Detailed Plan: Standalone React Web UI for Kubernetes with KubeVirt and ClusterAPI

1. Executive Summary

1.1 Project Overview

1.1.1 Purpose and Scope This project delivers a **comprehensive, standalone web-based user interface** built with React that unifies the management of virtualized infrastructure and Kubernetes cluster lifecycle operations. The UI targets organizations operating **strictly on-premises Kubernetes management clusters** equipped with **KubeVirt** for virtualization and **ClusterAPI with CAPK (Cluster API Provider Kubernetes)** for declarative cluster provisioning. The fundamental value proposition is eliminating the operational fragmentation of managing VMs and Kubernetes clusters through disparate tools, replacing command-line complexity with intuitive, web-based workflows.

The scope encompasses **four interconnected functional domains**: multi-tenant resource visualization with namespace-scoped access control; complete virtual machine lifecycle management from image upload through console access; self-service Kubernetes cluster provisioning via CAPI+CAPK's nested virtualization model; and CDI-based image storage with upload, import, and clone capabilities. The UI abstracts Kubernetes API complexity while preserving full declarative power, enabling both novice users through guided wizards and experts through direct YAML manipulation.

Unlike cloud-native management consoles, this solution must address **on-premises constraints**: air-gapped or limited connectivity environments, heterogeneous storage backends without cloud automation, manual network configuration, and self-managed high availability. The architecture prioritizes **security through isolation**—no cluster credentials reach browser clients, all operations transit through an authenticated proxy layer, and multi-tenancy enforcement leverages native Kubernetes RBAC rather than parallel permission systems.

1.1.2 Target Environment: On-Premises Management Cluster The deployment target is a **hardened Kubernetes management cluster** that serves as the sole control plane for all virtualized workloads and provisioned clusters. This cluster hosts the complete operational stack: **KubeVirt controllers** (virt-api, virt-controller, virt-handler DaemonSet, virt-launcher pods), **ClusterAPI core controllers** (capi-controller-manager, cabpk-bootstrap-kubeadm, kcp-control-plane-kubeadm), **CAPK infrastructure provider**, and **CDI components** (cdi-controller, cdi-uploadproxy, cdi-apiserver, cdi-operator). The management cluster's availability is paramount—its failure impacts all tenant workloads and provisioned clusters.

On-premises architectural imperatives shape every design decision:

Constraint	Implication for UI Design
No cloud load balancers	Integration with MetalLB, keepalived, or hardware load balancers for API server exposure and ingress
Heterogeneous storage	Explicit storage class selection, CDI storage profile awareness, and backend-specific optimization exposure

Constraint	Implication for UI Design
Air-gapped deployments	Offline container registry configuration, bundled Helm charts, and self-contained installation procedures
Custom PKI infrastructure	Certificate management interfaces for internal CAs, TLS termination configuration, and trust chain validation
Network segmentation	Multus CNI integration for VM networks, VLAN-aware network attachment definitions, and explicit connectivity validation

The management cluster requires **careful capacity planning**: control plane nodes need substantial CPU, memory, and I/O for etcd performance under the combined load of KubeVirt VM scheduling, ClusterAPI reconciliation, and CDI import operations. Worker nodes must accommodate both the virt-launcher pods for running VMs and the nested Kubernetes clusters created through CAPK. A **minimum production configuration** specifies three control plane nodes with 8+ CPU cores, 32GB+ RAM, and fast SSD storage for etcd, with worker nodes scaled according to anticipated VM density and cluster provisioning workload.

1.1.3 Core Technologies: React, KubeVirt, ClusterAPI, CAPK The technology stack unifies **modern frontend development** with **cloud-native infrastructure management**:

Layer	Technology	Role
Frontend	React 18+ with TypeScript	Component-based UI with compile-time type safety
State Management	Redux Toolkit + RTK Query	Predictable state, server cache synchronization, optimistic updates
Backend	Node.js/Express or Go	API proxy, authentication, Kubernetes client operations
Virtualization	KubeVirt v1.0+	VM lifecycle, live migration, console access via CRDs
Cluster Management	ClusterAPI v1beta1+	Declarative cluster provisioning with Machine-based scaling

Layer	Technology	Role
Infrastructure Provider	CAPK (Cluster API Provider Kubernetes)	Nested cluster provisioning as KubeVirt VMs
Storage	CDI v1.55+	Image import, upload, DataVolume provisioning

KubeVirt's architecture merits particular attention: the **virt-api server** provides HTTP API admission and validation; **virt-controller** manages cluster-wide VM orchestration; **virt-handler** runs as a DaemonSet for node-level coordination; and **virt-launcher** pods encapsulate individual QEMU/KVM processes (starlingx.io) . This pod-based virtualization model enables Kubernetes-native scheduling, network policies, and resource quotas to apply uniformly to VMs and containers.

CAPK's unique value lies in its **Kubernetes-in-Kubernetes provisioning pattern**: rather than requiring external cloud APIs or bare-metal provisioning tools, CAPK creates target cluster nodes as **KubeVirt VMs running on the management cluster itself** (TFiR) . This architecture delivers exceptional resource density, rapid provisioning (minutes rather than hours), and operational consistency —all control planes and worker nodes are themselves Kubernetes-scheduled workloads, subject to the same monitoring, logging, and governance as application workloads.

1.2 Key Capabilities

1.2.1 Multi-Tenant Resource Visualization The multi-tenancy implementation provides **each tenant with a comprehensive, filtered view** of their allocated resources across all infrastructure types —Kubernetes native resources (Pods, Services, ConfigMaps), KubeVirt VMs and DataVolumes, and CAPI-managed clusters—while **strictly preventing cross-tenant data leakage**. The visualization layer implements **namespace-scoped isolation** as the primary boundary, with each tenant operating within designated namespaces and RBAC enforcing visibility restrictions.

Critical visualization requirements:

- **Hierarchical resource relationships**: Clear presentation of how Cluster resources contain MachineDeployments, which manage MachineSets and individual Machines, with KubeVirt VMs as the underlying infrastructure
- **Real-time status aggregation**: Watch-based updates reflecting current state without manual refresh, with optimistic UI updates for user-initiated actions
- **Cross-cutting operational views**: Events, logs, and metrics filtered by tenant scope with correlation across resource types
- **Capacity and quota awareness**: Visual indicators of resource consumption against limits, with proactive warnings for approaching constraints

The tenant context pervades every UI interaction: navigation, resource lists, creation workflows, and search operations all respect the current tenant scope. **Administrative users** with cross-tenant per-

missions receive elevated views with clear visual distinction of resource ownership, preventing accidental operations in the wrong tenant context.

1.2.2 Virtual Machine Lifecycle Management VM lifecycle management encompasses **complete operational control** from initial provisioning through retirement:

Phase	UI Capabilities
Provisioning	Template selection, custom configuration wizard, cloud-init/sysprep injection, resource allocation (CPU, memory, huge pages, GPU)
Runtime	Power operations (start, stop, restart, pause), live migration, hot-plug disk/network attachment, snapshot/restore
Monitoring	Real-time metrics (CPU, memory, disk I/O, network), event correlation, log streaming from virt-launcher and guest agent
Access	VNC console via noVNC, serial console fallback, SSH key injection and access information
Decommissioning	Graceful shutdown with data preservation options, cascading deletion of associated resources

The **runStrategy field** in KubeVirt VirtualMachine resources determines automatic lifecycle behavior: **Always** maintains running state with automatic restart; **RerunOnFailure** restarts only on non-zero exit; **Manual** requires explicit user action; **Halted** keeps stopped regardless of spec (KubeVirt.io) . The UI must clearly communicate these semantics and their implications for operational behavior.

1.2.3 Cluster Provisioning via CAPI+CAPK The cluster creation capability enables **self-service Kubernetes cluster provisioning** with the unique characteristics of CAPK's nested virtualization:

- **Rapid provisioning**: New clusters available in 5-15 minutes versus hours for traditional infrastructure
- **Resource efficiency**: Control plane and worker nodes share management cluster infrastructure
- **Complete lifecycle**: Scaling, upgrades, and deletion through declarative API operations
- **Full Kubernetes conformance**: CAPK-provisioned clusters pass CNCF conformance tests

The creation workflow captures: **Kubernetes version** selection with upgrade path visualization; **control plane topology** (single-node for development, 3-node HA for production); **worker node pools** with autoscaling configuration; **network parameters** (CNI selection, pod/service CIDR allocation); and **CAPK-specific options** (VM resource templates, storage classes, network attachment definitions). Post-creation, the UI provides **kubeconfig retrieval**, cluster health monitoring, and integrated kubectl access.

1.2.4 CDI-Based Image Upload and Storage The **CDI Store** provides centralized VM image management with multiple acquisition pathways:

Source	Mechanism	Use Case
Direct upload	Browser → cdi-uploadproxy with chunked transfer	Custom images, air-gapped environments
HTTP/HTTPS import	CDI importer pod downloads from URL	Public image repositories, internal web servers
Registry import	Container image with embedded disk	CI/CD pipelines, versioned image distribution
PVC clone	Copy from existing DataVolume	Rapid provisioning from golden images
Snapshot restore	From VolumeSnapshot	Disaster recovery, point-in-time recovery

Upload experience requirements include: resumable transfers for multi-gigabyte files; progress indication with transfer rate and ETA; format validation (RAW, QCOW2, VMDK, ISO) with automatic conversion; and integrity verification through checksums. Storage optimization features expose **preallocation options** (sparse vs. full) and **storage profile-aware defaults** based on target storage class capabilities.

---

2. System Architecture

2.1 High-Level Architecture Pattern

2.1.1 React Frontend with Dedicated API Backend The architectural foundation mandates **strict separation between browser-based frontend and cluster-facing backend**. This pattern addresses fundamental security constraints: **Kubernetes service account tokens must never reach browser clients**, and **direct browser-to-API-server communication is blocked by CORS policies** (amlanscloud.com) . The React SPA communicates exclusively with a **same-origin backend API** that maintains authenticated, authorized connections to the Kubernetes control plane.

Frontend responsibilities: render interactive UI components; manage application state and user preferences; handle optimistic updates and loading states; establish WebSocket connections for real-time updates. **Backend responsibilities**: authenticate users against external identity providers; validate and authorize all requests; forward approved operations to Kubernetes API with appropriate impersonation; transform responses for frontend optimization; manage long-running operation polling and event streaming.

This separation enables **independent scaling and deployment**: the frontend builds to static assets served by any web server or CDN, while the backend scales horizontally based on API load and connection density. The backend's **Kubernetes client maintains persistent connections** with connection pooling, watch stream multiplexing, and automatic reconnection for resilience.

2.1.2 API Proxy/Gateway for Secure Cluster Communication The API proxy layer implements **defense-in-depth security** for cluster access:

Security Control	Implementation
Authentication	OIDC/OAuth2 flow with PKCE for SPAs; JWT validation with JWKS endpoint consultation; session management with httpOnly cookies or short-lived access tokens
Authorization	Kubernetes RBAC through user/group impersonation; SelfSubjectAccessReview for UI permission reflection
Request validation	Schema validation against OpenAPI; size limits; rate limiting per user and operation type
Audit logging	Comprehensive request/response logging with user attribution for compliance
WebSocket security	Authentication at upgrade handshake; connection limits; idle timeout; protocol validation for VNC streams

The proxy **must handle Kubernetes subresource access patterns** that differ from standard REST: WebSocket upgrade for exec, port-forward, and VNC; SPDY framing for legacy streaming; and binary protocol translation for noVNC compatibility (Github) . For VNC specifically, the proxy bridges the browser's RFC 6455 WebSocket connection to Kubernetes' SPDY-based subresource stream, maintaining authentication context throughout the persistent connection.

2.1.3 Microservices-Inspired Component Design While initially deployed as a monolithic application, the backend structures internal **service boundaries** for future decomposition:

Service Domain	Responsibilities	Extraction Trigger
Resource Service	CRUD operations on all Kubernetes resources; caching; field selection	High read load requiring independent scaling
Console Service	VNC/serial WebSocket proxying; connection lifecycle management	High connection count from many concurrent consoles
Upload Service	CDI upload coordination; chunked transfer handling; progress streaming	Large file throughput requiring dedicated storage connectivity

Service Domain	Responsibilities	Extraction Trigger
Cluster Service	CAPI provisioning workflows; async operation coordination; kubeconfig generation	Complex state machine requiring specialized reliability
Tenant Service	Multi-tenancy management; namespace provisioning; quota enforcement	Organizational scaling requiring delegated administration

Each service boundary has **defined API contracts** enabling independent testing and eventual extraction. Shared concerns—authentication middleware, structured logging, metrics emission, circuit breakers—are implemented as **cross-cutting infrastructure** rather than duplicated per service.

2.2 Component Layers

2.2.1 Presentation Layer: React UI Components The React application organizes components by **feature domains** with clear separation of concerns:

Foundation layer: Design system implementation (PatternFly or Material-UI) with theme customization, typography, spacing, and color tokens. Shared primitives—Button, Input, Select, Table, Modal, Form—provide consistent behavior and accessibility compliance.

Resource components: Generic `ResourceList<T>`, `ResourceDetail<T>`, `ResourceForm<T>` components parameterized by resource type, with specialized extensions for KubeVirt and CAPI resources. These handle Kubernetes-specific patterns: label selectors, field selectors, owner reference navigation, and condition aggregation.

Feature components: Domain-specific implementations for VM management (VncConsole, VmActions, DiskConfiguration), cluster provisioning (ClusterWizard, MachineSetScaling, UpgradeProgress), and CDI operations (ImageUpload, DataVolumeStatus, StorageClassSelector).

State architecture: Hybrid approach combining **Redux Toolkit** for global state (auth, tenant context, preferences) with **RTK Query** for server state (resource caching, background refetching, optimistic updates) and **React hooks** for local component state. WebSocket events from Kubernetes watches dispatch Redux actions for cache invalidation and real-time updates.

2.2.2 API Abstraction Layer: Backend Service (Node.js/Express or Go) Technology selection involves deliberate tradeoffs:

Criterion	Node.js/Express	Go
Development velocity	Excellent—JavaScript/TypeScript ecosystem, hot reload, shared types with frontend	Moderate —static typing overhead, explicit error handling

Criterion	Node.js/Express	Go
Kubernetes client maturity	Good—`@kubernetes/client-node` with TypeScript, occasional lag behind `client-go`	Excellent—`client-go` is reference implementation, immediate API updates
Concurrency model	Event loop with async/await; clustering for multi-core	Goroutines with excellent performance for high-connection scenarios
WebSocket handling	Good with `ws` library; memory management for long connections	Excellent—goroutine-per-connection scales efficiently
Deployment	Container with Node runtime; larger image size	Static binary; minimal container image
Ecosystem alignment	Frontend team familiarity; npm package abundance	Kubernetes community conventions; operator ecosystem

Recommendation: **Node.js/Express for initial development** with migration path to Go if performance testing reveals bottlenecks in VNC proxying or high-connection scenarios. The `@kubernetes/client-node` library provides adequate functionality for most operations, with the `kubernetes-client/javascript` informer implementation supporting efficient resource watching (Kubernetes) .

The backend implements **layered architecture**: HTTP handlers for request/response concerns; service layer for business logic and transaction coordination; repository layer for Kubernetes API interaction; infrastructure layer for logging, metrics, and configuration. **Dependency injection** enables testable components with mock Kubernetes clients for unit testing.

2.2.3 Kubernetes Integration Layer: CRD Controllers and Clients Direct Kubernetes integration requires **type-safe clients for multiple API groups**:

API Group	Resources	Client Source
`kubevirt.io/v1`	VirtualMachine, VirtualMachineInstance, VirtualMachineInstanceMigration	`@kubevirt-ui/kubevirt-api` generated types (npm)
`cdi.kubevirt.io/v1beta1`	DataVolume, DataSource, StorageProfile	`@kubevirt-ui/kubevirt-api` or manual definition
`cluster.x-k8s.io/v1beta1`	Cluster, Machine, MachineSet, MachineDeployment, MachineHealthCheck	OpenAPI generation from CAPI CRDs
`infrastructure.cluster.x-k8s.io/v1alpha1`	KubevirtCluster, KubevirtMachine, KubevirtMachineTemplate	CAPK-specific generation
`controlplane.cluster.x-k8s.io/v1beta1`	KubeadmControlPlane	CAPI control plane provider

Informer-based caching maintains local state for frequently accessed resources, with watch streams providing real-time updates. The informer pattern—listing existing resources then watching for changes—minimizes API server load while ensuring cache consistency. **Direct API calls** for write operations and situations requiring strong consistency bypass the cache.

2.3 Security Architecture

2.3.1 Authentication Proxy Pattern The authentication flow implements **OIDC Authorization Code Flow with PKCE** for single-page application security:

1. User initiates login; backend redirects to identity provider with PKCE code challenge
2. Identity provider authenticates user and redirects to backend with authorization code
3. Backend exchanges code for tokens (access token, refresh token, ID token) with PKCE verifier
4. Backend establishes session (encrypted httpOnly cookie or short-lived JWT) and redirects to UI
5. Subsequent requests present session cookie; backend validates and extracts user identity

Session management includes: token refresh before expiration with rotation; concurrent session limits per user; explicit logout with token revocation; and session timeout with configurable idle and absolute limits.

2.3.2 Service Account Impersonation All Kubernetes API operations execute with **user impersonation** to enforce RBAC consistently:

```
Impersonate-User: alice@example.com
Impersonate-Group: tenant-engineering
```

```
Impersonate-Group: platform-users
```

The backend's service account requires **impersonation permissions**:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: ui-proxy-impersonator
rules:

- apiGroups: [""]

  resources: ["users", "groups", "serviceaccounts"]
  verbs: ["impersonate"]
```

This pattern ensures **audit logs reflect actual user identity** and **authorization decisions use tenant-specific RBAC bindings** rather than the proxy's elevated permissions. The mapping from external identity (OIDC claims) to Kubernetes user/group identifiers is configurable for organizational alignment.

2.3.3 RBAC Integration for Multi-Tenancy **Tenant isolation through namespace-scoped RBAC**:

Role	Scope	Permissions
`tenant-admin`	Namespace	Full resource management within tenant namespaces; user invitation; quota viewing
`tenant-editor`	Namespace	Create, update, delete VMs, DataVolumes, clusters; read all resources
`tenant-viewer`	Namespace	Read-only access to all resources; console access where explicitly granted
`platform-admin`	Cluster	Full cluster access; tenant provisioning; infrastructure configuration

The UI queries **SelfSubjectRulesReview** to determine effective permissions and adapts interface elements accordingly—hiding unavailable actions, disabling creation buttons for quota-exhausted tenants, and presenting appropriate error messages for authorization failures (headlamp.dev) .

2.3.4 Same-Origin Policy Handling for WebSocket Connections VNC console access requires **WebSocket proxying through the same-origin backend** (Github) :

```
Browser ↔ wss://ui.example.com/api/vnc/{namespace}/{vmi-name} ↔ Kubernetes API wss://.../vnc
```

The backend handles: **protocol upgrade** with authentication validation; **bidirectional frame forwarding** between RFC 6455 (browser) and SPDY (Kubernetes) framing; **connection lifecycle management** with heartbeat, idle timeout, and graceful shutdown; and **security controls** including connection limits per user/VM and optional session recording.

---

3. Frontend Design (React)

3.1 Core UI Framework

3.1.1 React with TypeScript for Type Safety TypeScript configuration enforces **strict type checking** with `noImplicitAny`, `strictNullChecks`, and `strictFunctionTypes`. Generated types from `@kubevirt-ui/kubevirt-api` provide accurate interfaces for KubeVirt and CDI resources, with custom extensions for UI-specific structures like form state and API responses.

Type generation workflow: OpenAPI specifications from target cluster → `openapi-typescript` generation → manual refinement for discriminated unions and generic patterns. Shared type definitions between frontend and backend (via npm workspace or git submodule) ensure API contract consistency.

3.1.2 State Management: Redux with Async Middleware (Thunk/Saga) **State architecture by concern**:

State Category	Management	Persistence
Authentication	Redux with secure cookie/ httpOnly JWT	Server-side session
Tenant/Namespace context	Redux with URL synchronization	URL params, localStorage preference
Kubernetes resources	RTK Query with normalized cache	In-memory with revalidation
UI state (selections, modals)	React hooks + Zustand	None (ephemeral)
Form state	React Hook Form	Draft auto-save to localStorage

Complex async flows (cluster provisioning with multiple dependent resources) benefit from **Redux-Saga** for declarative effect orchestration: parallel API calls, retry with backoff, cancellation on user abort, and rollback on partial failure.

3.1.3 UI Component Libraries: PatternFly or Material-UI **PatternFly is strongly recommended** for this infrastructure management domain:

- **Enterprise focus**: Designed for OpenShift and administrative interfaces
- **Kubernetes-native patterns**: Resource tables, status indicators, wizard flows
- **Accessibility**: WCAG 2.1 AA compliance out-of-box

- **Stability**: Red Hat commercial backing with predictable release cadence

Material-UI remains viable for organizations with existing design system investment, requiring more customization for data-dense administrative patterns.

3.2 KubeVirt-Specific Components

3.2.1 Virtual Machine List and Detail Views The **VM list** implements virtualized scrolling for large deployments with columns: name (with status icon), namespace, **status** (Running/Stopped/Provisioning/Error with color coding), node, IP address, CPU/memory allocation, creation time, and row actions. **Status aggregation** combines VirtualMachine `spec.runStrategy` with VirtualMachineInstance `status.phase` for accurate representation (KubeVirt.io) .

The **detail view** uses tabbed navigation: **Overview** (status summary, resource charts, quick actions), **Details** (editable specification with field-level help), **Events** (filtered, searchable event stream), **Console** (VNC/serial integration), **Disks** (attached storage with usage), **Network** (interfaces with IP/MAC), and **YAML** (full manifest with edit capability).

3.2.2 VM Console Integration: noVNC for VNC Access **noVNC integration architecture**:

Component	Responsibility
`VncConsole` React component	noVNC initialization, connection state UI, toolbar controls
Backend WebSocket proxy	Authentication, Kubernetes API bridging, protocol translation
noVNC core (`@novnc/novnc`)	RFB protocol, canvas rendering, input handling

Critical implementation details: WebSocket URL construction with backend proxy path; connection state machine (connecting → connected → disconnected → error); keyboard capture mode for special keys (Ctrl+Alt+Del, F-keys); quality adaptation for bandwidth constraints; and automatic reconnection with exponential backoff (Github) (CSDN 博客) .

3.2.3 VM Action Components: Start, Stop, Restart, Delete **Action semantics and confirmation patterns**:

Action	KubeVirt Mechanism	Confirmation Required	Notes
Start	Set `spec.running: true` or `runStrategy: Always`	No	Idempotent; no-op if already running

Action	KubeVirt Mechanism	Confirmation Required	Notes
Stop (graceful)	`virtctl stop` or `runStrategy: Halted`	Optional	ACPI shutdown signal; configurable timeout
Stop (force)	Delete VirtualMachineInstance	Yes	Immediate termination; data corruption risk
Restart	Stop then start sequence	No	Optimized for minimal downtime
Delete	Delete VirtualMachine with propagation	**Strong confirmation**	Cascade options for DataVolumes, Services

Optimistic updates immediately reflect action in UI, with rollback on API error. **Progress tracking** for long operations (live migration, large DataVolume creation) uses Kubernetes watch events with visual progress indicators.

3.2.4 Disk and Network Configuration Forms **Disk configuration** supports multiple volume types with dynamic form fields:

Volume Type	Configuration Fields	Hot-Plug Support
DataVolume	Source (blank/HTTP/registry/clone/upload), size, storage class	Yes
PersistentVolumeClaim	Existing PVC selector, read-only option	Yes
ContainerDisk	Image reference, pull policy	No (ephemeral)
ConfigMap/Secret	ConfigMap/Secret name, volume path	No
EmptyDisk	Size (ephemeral scratch)	No

Network configuration with CNI integration: primary pod network (masquerade or bridge), Multus

secondary networks (NetworkAttachmentDefinition selection), SR-IOV resource allocation, and MAC address assignment (auto or manual).

3.3 ClusterAPI Components

3.3.1 Cluster List and Status Dashboard **Cluster status aggregation** from multiple CAPI resources:

Resource	Key Status Fields	UI Presentation
`Cluster`	`status.phase`, `status.conditions`	Overall health indicator, phase badge
`KubeadmControlPlane`	`status.replicas`, `status.readyReplicas`, `status.version`	Control plane node count, version
`MachineDeployment`	`status.replicas`, `status.readyReplicas`, `status.unavailableReplicas`	Worker pool scaling status
`KubevirtCluster`	`status.ready`, `status.failureReason`	Infrastructure readiness

Dashboard visualization: fleet-wide cluster health summary; recent provisioning activity; version distribution across clusters; and capacity utilization of underlying KubeVirt infrastructure.

3.3.2 Cluster Creation Wizard **Wizard step progression** with validation gates:

1. **Basics**: Name, namespace, Kubernetes version (with support lifecycle indication)
2. **Control Plane**: Topology (single/HA), machine template (CPU/memory/disk), replicas
3. **Worker Pools**: Pool name, replicas (min/max for autoscaling), machine template, labels/taints
4. **Network**: CNI selection (Calico/Cilium/Flannel), pod CIDR, service CIDR, API server access (LoadBalancer/NodePort)
5. **CAPK Options**: VM node labels, affinity rules, dedicated infrastructure flag
6. **Review**: Generated resource manifest preview, estimated resource consumption

ClusterClass integration (k8s.io) : When available, present templated configurations with variable substitution UI rather than raw resource editing.

3.3.3 Machine and MachineSet Visualization **Machine lifecycle visualization**: Pending → Provisioning → Provisioned → Running → Deleting, with infrastructure-specific details for CAPK (VM name, node assignment, pod status). **Rolling update progress** for MachineDeployments shows old/new revision counts, surge configuration, and per-machine update status.

3.3.4 Infrastructure Provider Status (CAPK) **CAPK-specific operational views**: KubevirtCluster resource with VM network configuration; KubevirtMachineTemplate library for reusable node specifications; and **nested cluster topology visualization** showing the relationship between management cluster VMs and the workload clusters they host.

3.4 CDI Store Interface

3.4.1 DataVolume List and Management **DataVolume status phases** with visual indicators: Pending → ImportScheduled → ImportInProgress (with percentage) → Succeeded/Failed. **Source-specific details**: HTTP URL with link validation; registry image reference; upload token expiration; clone source with navigation.

Management operations: Clone (create new DataVolume from existing); Expand (increase capacity where supported); and Delete with usage checking (warn if attached to running VM).

3.4.2 Image Upload Component with Progress Tracking **Upload implementation with resumable transfer**:

Feature	Implementation
Chunk size	10-100MB configurable based on network reliability
Progress tracking	XMLHttpRequest progress events → Redux state → progress bar
Resume	HTTP Range header support with server-side chunk tracking
Validation	Client-side format detection (magic numbers), server-side checksum
Error handling	Per-chunk retry with exponential backoff; user-initiated resume

CDI upload proxy integration (Github) : Backend generates UploadTokenRequest → extracts token → constructs upload URL → frontend POSTs to backend proxy → backend streams to cdi-uploadproxy with token authentication.

3.4.3 PVC and Storage Class Integration **Storage class selection** with capability indication: provisioner type, volume binding mode (Immediate/WaitForFirstConsumer), allowed topologies, and CDI storage profile defaults. **PVC visualization** shows actual capacity vs. request, access mode, volume mode (Filesystem/Block), and DataVolume ownership relationship.

3.5 Multi-Tenancy UI Patterns

3.5.1 Tenant Context Switcher **Persistent header component** with: current tenant/namespace display with color coding; dropdown of accessible tenants with search; recent/favorite quick access; and **unsaved changes warning** on context switch.

3.5.2 Namespace-Scoped Resource Views **Consistent namespace filtering** across all resource types, with "All namespaces" option for users with cluster-scoped permissions. **URL-encoded namespace state** enables bookmarking and sharing of filtered views.

3.5.3 Role-Based Menu and Action Visibility **Dynamic UI adaptation** based on SelfSubjectRulesReview results: hide menu items for unauthorized resource types; disable action buttons for missing permissions; and show **permission explanation tooltips** for disabled actions.

4. Backend API Service

4.1 API Proxy Responsibilities

4.1.1 Kubernetes API Request Forwarding **Request transformation pipeline**: path rewriting (UI-friendly to Kubernetes API), query parameter mapping (pagination, filtering), header injection (impersonation, content-type), and body transformation (field mapping, default injection). **Response transformation**: field selection for reduced payload, format normalization, and error message enhancement.

4.1.2 Authentication Token Validation **JWT validation flow**: signature verification with JWKS caching; claim validation (exp, iss, aud, sub); and group extraction for impersonation. **Session alternatives**: encrypted cookie with server-side session store for traditional web sessions; or short-lived access tokens with refresh rotation for SPA patterns.

4.1.3 Request Transformation and Validation **Schema validation** using OpenAPI specifications with AJV (Node.js) or hand-rolled validators (Go). **Business rule validation**: cross-field consistency, quota pre-checking, and resource reference existence.

4.1.4 Response Caching and Optimization **Informer-based caching** with watch-driven invalidation for list responses. **ETag generation** for conditional requests. **Field selection** support via query parameters to reduce payload sizes.

4.2 Technology Stack Options

Aspect	Node.js/Express	Go
Recommended for	Rapid development, team JS expertise, full-stack TypeScript	High-scale production, performance-critical VNC proxying, Kubernetes ecosystem alignment
Key libraries	`@kubernetes/client-node`, `express`, `ws`, `jsonwebtoken`	`client-go`, `gin` or `echo`, `gorilla/websocket`
Deployment	`node:20-alpine` container, ~200MB	Static binary in `scratch` or `distroless`, ~20MB

4.3 Core API Endpoints

Endpoint	Method	Description
`/api/v1/:resource`	GET, POST	List and create namespaced resources
`/api/v1/:resource/:name`	GET, PUT, PATCH, DELETE	Resource CRUD with optimistic concurrency
`/api/v1/:resource/:name/watch`	WebSocket	Real-time resource change stream
`/api/vnc/:namespace/:name`	WebSocket	VNC console proxy to KubeVirt subresource
`/api/upload`	POST	CDI image upload with multipart handling
`/api/clusters`	POST	Cluster creation with CAPI resource generation
`/api/tenants`	GET, POST	Tenant listing and provisioning (admin)
`/api/auth/*`	Various	OIDC flow endpoints, session management

---

5. Kubernetes Integration

5.1 Core Kubernetes Resources

5.1.1 Namespace Management for Multi-Tenancy **Namespace provisioning workflow**: creation with tenant labels; ResourceQuota and LimitRange application; default NetworkPolicy installation; and RBAC RoleBinding for tenant members.

5.1.2 Pod, Service, ConfigMap, Secret Visualization **Infrastructure context**: virt-launcher pods with resource usage and log access; Services exposing VM ports with LoadBalancer status; ConfigMaps/Secrets for cloud-init and application configuration with usage tracking.

5.1.3 Event and Log Streaming **Event aggregation** with field selector filtering by involved object. **Log streaming** via Kubernetes API with follow option, integrated into VM detail views for virt-launcher and guest agent logs.

5.2 KubeVirt Integration

5.2.1 VirtualMachine and VirtualMachineInstance CRDs **Resource relationship**: VirtualMachine as user-managed desired state; VirtualMachineInstance as system-managed runtime realization. **UI presentation**: VM-centric with VMI details accessible for troubleshooting (starlingx.io) .

5.2.2 VNC Subresource Access Endpoint: `wss://kubernetes-api/apis/subresources.kubevirt.io/v1/namespaces/{ns}/virtualmachineinstances/{name}/vnc` (CSDN 博客) . **Proxy requirement**: Backend-mediated access for same-origin compliance and authentication.

5.2.3 VM Lifecycle State Management State machine visualization: combining `spec.runStrategy` intent with `status.phase` reality, showing transitions and blocking conditions.

5.2.4 Hot-Plug Disk and Network Attachments Dynamic attachment via KubeVirt addVolume/removeVolume APIs, with UI indication of hot-plugged vs. template-defined resources.

5.3 Containerized Data Importer (CDI) Integration

5.3.1 DataVolume CRD for Storage Provisioning Unified abstraction: PVC creation + data population in single resource, with source-type-specific progress tracking (Github) .

5.3.2 Upload Token Generation and Validation Security mechanism: Short-lived, single-use tokens binding upload to specific DataVolume, generated via UploadTokenRequest API (Github) .

5.3.3 Image Import Workflows

Source	CDI Mechanism	UI Pattern
HTTP/HTTPS	Importer pod download	URL input with validation
Registry	Container image pull	Image reference with pull secret
Upload	cdi-uploadproxy	Drag-and-drop with progress
PVC clone	Volume cloning	Source PVC selection

5.3.4 Storage Profile and PVC Management Intelligent defaults: StorageProfile resources define optimal access mode, volume mode, and filesystem type per storage class.

5.4 ClusterAPI (CAPI) and CAPK Integration

5.4.1 Cluster CRD: Declarative Cluster Definitions Spec structure: control plane reference, infrastructure reference, cluster network configuration. **Status tracking**: phase, conditions, and control plane endpoint for kubeconfig generation.

5.4.2 Machine, MachineSet, and MachineDeployment Resources Scaling and update orchestration: declarative replica counts with rolling replacement strategies.

5.4.3 Bootstrap and Control Plane Provider Integration Kubeadm integration: cloud-init generation, certificate management, and etcd coordination.

5.4.4 CAPK-Specific: Kubernetes-in-Kubernetes Provisioning Unique value proposition: Rapid, resource-efficient cluster provisioning with VMs as the infrastructure abstraction (TFiR) .

5.4.5 ClusterClass for Templated Cluster Creation Parameterized templates: Variable substitution for standardized cluster configurations with organizational defaults (k8s.io) .

---

6. Multi-Tenancy Implementation

6.1 Tenant Isolation Strategies

6.1.1 Namespace-Per-Tenant Model **Primary isolation boundary** with optional multiple namespaces per tenant for environment separation (dev/staging/prod).

6.1.2 Custom Resource Quotas and Limits

Resource Type	Quota Mechanism	UI Visibility
Compute	ResourceQuota (requests/limits)	Real-time usage gauge
Storage	ResourceQuota (PVC count, total capacity)	Per-storage-class breakdown
Object count	ResourceQuota (count/)	Trending and alerts

6.1.3 Network Policies for Inter-Tenant Segregation **Default deny** with explicit allow rules for required cross-tenant communication.

6.2 Authentication and Authorization

6.2.1 SSO Integration: OIDC, LDAP, or SAML **OIDC preferred** for modern deployments with PKCE for SPA security. LDAP/SAML for legacy enterprise integration.

6.2.2 Kubernetes RBAC Mapping for Tenant Users **Group claim mapping** from identity provider to Kubernetes groups for role assignment.

6.2.3 Custom UserService API for Tenant Management **Self-service capabilities**: Member invitation, role assignment, and quota increase requests with approval workflow.

6.2.4 Token-Based API Access with Short-Lived Credentials **Access token lifetime**: 15-60 minutes; refresh token rotation with revocation support.

6.3 Resource Visibility Controls

6.3.1 Tenant-Scoped API Filtering **Server-side enforcement** with namespace filtering on all list operations and resource reference validation.

6.3.2 Cross-Tenant Resource Sharing (Optional) **Controlled sharing**: Shared DataVolumes for golden images; explicit network policy exceptions.

6.3.3 Audit Logging for Compliance **Comprehensive logging**: Authentication events, resource modifications, and administrative actions with structured format for SIEM integration.

---

7. Key Feature Implementation

7.1 CDI Store: Image Upload and Management

7.1.1 Upload Proxy Integration: `cdi-uploadproxy` Service **Service exposure**: LoadBalancer or Ingress with TLS termination; backend-mediated access for authentication (Github) .

7.1.2 Client-Side Upload with Progress Indicators **Resumable chunked upload** with progress streaming via Server-Sent Events or WebSocket.

7.1.3 Supported Formats: RAW, QCOW2, VMDK, ISO **Automatic format detection** and conversion to storage-optimized representation.

7.1.4 DataVolume Preallocation and Storage Optimization **Preallocation options**: Sparse (thin-provisioned) for efficiency; full for performance-critical workloads.

7.2 Virtual Machine Deployment

7.2.1 VM Template Library **Curated templates**: Operating system base images with pre-configured resource profiles and initialization scripts.

7.2.2 Cloud-Init and Sysprep Integration **Initialization data injection**: cloud-init YAML for Linux; unattend.xml for Windows via ConfigMap/Secret.

7.2.3 Resource Scheduling and Node Affinity **Scheduling controls**: Node selectors, affinity/anti-affinity rules, and dedicated node pools for workload isolation.

7.2.4 Live Migration and High Availability **Live migration UI**: Initiation with progress tracking, automatic evacuation for node maintenance, and migration policy configuration.

7.3 VM Console Connection

7.3.1 noVNC Integration in React Components **Component library**: `react-vnc` or `react-vnc-display` with connection state management and quality controls (unpkg.com) (Github) .

7.3.2 WebSocket Proxy for Secure VNC Access **Backend-mediated connection** with authentication, protocol translation, and session management (Github) .

7.3.3 Serial Console Fallback Option **Text-mode access** for headless VMs and recovery scenarios when VNC is unavailable.

7.3.4 Clipboard and File Transfer Capabilities **Clipboard integration**: Bidirectional text copy/paste with security confirmation. File transfer via guest agent where available.

7.4 Target Cluster Creation (CAPI+CAPK)

7.4.1 Cluster Template Definition UI **ClusterClass visualization**: Variable schema with descriptive help and validation constraints.

7.4.2 Control Plane and Worker Node Sizing **Resource calculator**: Estimated management cluster consumption based on target cluster specifications.

7.4.3 Kubernetes Version Selection **Version compatibility**: Supported upgrade paths and deprecation warnings.

7.4.4 Network Configuration: CNI, Pod/Service CIDR **Conflict detection**: Validation against existing network allocations and management cluster networking.

7.4.5 Post-Provisioning Cluster Access Configuration **Kubeconfig retrieval**: Secure download with embedded credentials or external identity integration.

---

8. On-Premises Deployment Considerations

8.1 Infrastructure Requirements

8.1.1 Management Cluster Sizing and High Availability

Component	Minimum Production	Recommended
Control plane nodes	3	3-5
CPU per control plane	8 cores	16 cores
Memory per control plane	32 GB	64 GB
Storage per control plane	500 GB SSD	1 TB NVMe
Worker nodes	3	5+
Network	10 Gbps	25 Gbps

8.1.2 Storage Backend: Local, Ceph, or NFS

Backend	Characteristics	CDI Optimization
Local (LVM)	Highest performance, no shared storage	Clone via rsync; limited live migration
Ceph RBD	Distributed, snapshots, thin provisioning	Native cloning; efficient copy-on-write
NFS	Simple, widely supported	Server-side copy for clones; performance variability

8.1.3 Network Requirements: Load Balancers, Ingress **MetalLB** for LoadBalancer service type; **NGINX Ingress Controller** or **Traefik** for HTTP/HTTPS routing; **cert-manager** for TLS certificate automation.

8.2 Deployment Topology

8.2.1 Air-Gapped Installation Support **Offline requirements**: Pre-pulled container images in internal registry; Helm charts bundled; installation scripts with no external dependencies.

8.2.2 Offline Image Registry Configuration **Registry mirror configuration**: Image pull through to external registries with caching; or complete air-gap with manual image import.

8.2.3 Certificate Management for Internal TLS **Private CA integration**: cert-manager with internal issuer; or manual certificate distribution for strict environments.

8.3 Operational Concerns

8.3.1 Backup and Disaster Recovery **Critical data**: etcd snapshots for cluster state; PersistentVolume snapshots for VM data; and ClusterAPI resource exports for cluster definitions.

8.3.2 Upgrade Strategies for Management Components **Rolling upgrade pattern**: KubeVirt, CDI, CAPI, and CAPK upgrades with version compatibility matrix validation.

8.3.3 Monitoring and Alerting Integration **Prometheus/Grafana stack**: Metrics collection from all components; alerting rules for capacity, health, and performance anomalies.

---

9. Development and Implementation Roadmap

9.1 Phase 1: Foundation (Weeks 1-4)

Deliverable	Description
React + TypeScript project setup	Build tooling, linting, testing framework
Backend API proxy skeleton	Express or Go with health endpoints
Kubernetes client integration	Resource listing with informer caching
Basic resource table views	Pods, Namespaces with sorting/filtering

9.2 Phase 2: KubeVirt Integration (Weeks 5-8)

Deliverable	Description
VM list and detail views	Status aggregation, resource relationships
VM lifecycle actions	Start, stop, restart with optimistic updates
VNC console integration	noVNC component with WebSocket proxy
Disk and network forms	Dynamic configuration with validation

9.3 Phase 3: CDI and Storage (Weeks 9-12)

Deliverable	Description
DataVolume management interface	List, status, clone, expand operations
Image upload functionality	Chunked upload with progress, resume
Storage class integration	Selection with capability awareness

9.4 Phase 4: Cluster Provisioning (Weeks 13-16)

Deliverable	Description
CAPI resource visualization	Cluster, Machine, MachineSet status
Cluster creation wizard	Multi-step with validation and preview
CAPK-specific configuration	VM templates, network attachment

9.5 Phase 5: Multi-Tenancy and Hardening (Weeks 17-20)

Deliverable	Description
Tenant isolation implementation	Namespace provisioning, RBAC integration
SSO and RBAC integration	OIDC flow, permission-based UI adaptation
Security audit and penetration testing	Vulnerability assessment, remediation
Production readiness documentation	Runbooks, troubleshooting guides

---

10. Reference Implementations and Libraries

10.1 Existing Projects

Project	Technology	Relevance
kubevirt-manager	Angular	KubeVirt-specific UI patterns, VNC integration approach (Github)

Project	Technology	Relevance
Kubernetes Dashboard	Go/Angular	API proxy architecture, authentication patterns (cloudnativenow.com)
OpenShift Console	PatternFly/React	Enterprise Kubernetes UI design system, component patterns

10.2 Reusable Libraries

Library	Purpose	Source
`@kubevirt-ui/kubevirt-api`	TypeScript types for KubeVirt/ CDI CRDs	(npm)
`@kubernetes/client-node`	Official Kubernetes client for Node.js	(Kubernetes)
`noVNC`	Browser-based VNC client	(novnc.com)
`react-vnc` / `react-vnc-display`	React wrappers for noVNC	(unpkg.com) (Github)

10.3 Community Resources

Resource	URL	Content
Cluster API documentation	cluster-api.sigs.k8s.io	Architecture, quickstart, provider development
KubeVirt user guide	kubevirt.io/ user-guide	VM management, networking, storage
CDI upload documentation	github.com/ kubevirt/ containerized-data-importer	Upload API, storage profiles